

Open-Sora 2.0: Training a Commercial-Level Video Generation Model in \$200k

Open-Sora Team

HPC-AI Tech

Abstract

Video generation models have achieved remarkable progress in the past year. The quality of AI video continues to improve, but at the cost of larger model size, increased data quantity, and greater demand for training compute. In this report, we present Open-Sora 2.0, a commercial-level video generation model trained for only \$200k. With this model, we demonstrate that the cost of training a top-performing video generation model is highly controllable. We detail all techniques that contribute to this efficiency breakthrough, including data curation, model architecture, training strategy, and system optimization. According to human evaluation results and VBench scores, Open-Sora 2.0 is comparable to global leading video generation models including the open-source HunyuanVideo and the closed-source Runway Gen-3 Alpha. By making Open-Sora 2.0 fully open-source, we aim to democratize access to advanced video generation technology, fostering broader innovation and creativity in content creation. All resources are publicly available at: <https://github.com/hpcaitech/Open-Sora>.

1 Introduction

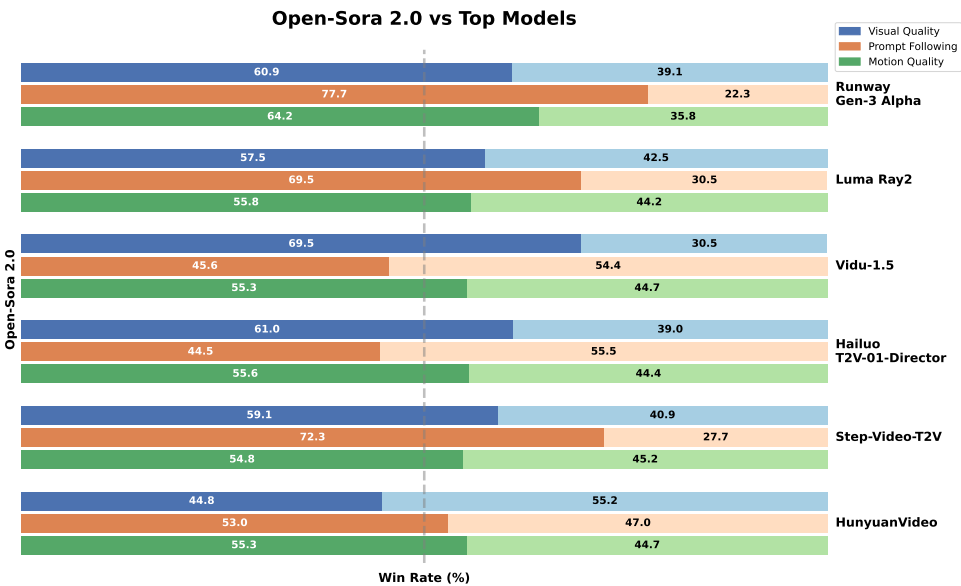


Figure 1: Human preference evaluation of Open-Sora 2.0 against other leading video generation models. Win rate represents the percentage of comparisons where our model was preferred over the competing model. The evaluation is conducted on 100 prompts carefully designed to cover three key aspects: 1) visual quality, 2) prompt adherence, and 3) motion quality. Results show that our model performs favorably against other top-performing models in all three aspects.

The past year has witnessed an explosion of video generation models. Since the emergence of OpenAI’s Sora [4] in February 2024, a series of video generation models—either open-source [16, 19, 22, 27, 44] or proprietary [26, 32, 35]—have appeared at an unprecedented pace, each striving to achieve "Sora-level" generation quality. While the quality of generated videos continues to improve, there is a clear trend toward rapid growth in model size, data quantity, and computing resources. Following the success of scaling large language models (LLMs) [15, 18], researchers are now applying similar scaling principles [19] to video generation, converging on similar model architectures and training techniques.

In this report, however, we show that a top-performing video generation model can be trained at a highly controlled cost, offering new insights into cost-effective training and efficiency optimization. We build Open-Sora 2.0, a commercial-level video generation model trained for only \$200k. As shown in Table 7, the training cost is 5-10 times lower than comparable models like MovieGen [32] and Step-Video-T2V [27] based on a fair comparison. This remarkable cost efficiency stems from our joint optimization of data curation, training strategy, and AI infrastructure—all of which are detailed in the following sections.

We show the human preference evaluation of Open-Sora 2.0 and other global leading video generation models in Figure 1. The models for comparison include proprietary models like Runway Gen-3 Alpha [35], Luma Ray2 [26] and open-source models like HunyuanVideo [19] and Step-Video-T2V [27]. We evaluate these models in three important aspects: 1) visual quality, 2) prompt adherence, and 3) motion quality. Despite the low cost of our model, it outperforms these top-performing models in at least two aspects out of the three, demonstrating its strong potential for commercial deployment.

This report is structured as follows. In Section 2, we elaborate on our data strategy, including our hierarchical data filtering system and annotation methods. Section 3 details our model architecture, covering both our novel Video DC-AE autoencoder design and the DiT architecture. Section 4 explores our cost-effective training methodology, which enables commercial-level quality at just \$200k. Section 5 presents our conditioning approaches, including image-to-video and motion control techniques. Section 6 outlines the system optimizations that maximize training efficiency, and Section 7 evaluates our model’s performance against leading video generation models. Finally, we conclude with a summary of our contributions and insights for future research.

2 Data

Our goal for data is to build a hierarchical data pyramid, catering to the requirement of the progressive training process. To this end, we develop a collection of filters that function distinctly from each other, aiming to tackle various types of data detections. By progressively strengthening the degree of filtration, we can obtain subsets of smaller sizes but higher purity and quality.

In Section 2.1, we first elaborate the data filtering system that consists of two main components: preprocessing and score filtering. Then, we introduce data annotation methods in Section 2.2. Finally, we present detailed statistics of the whole dataset in Section 2.3.

2.1 Data Filtering

The hierarchical data filtering system is illustrated in Figure 2. We begin by preprocessing raw videos into trainable video clips, then progressively apply a series of filters, ranging from loose to strict, to construct a structured data pyramid.

2.1.1 Preprocessing

The preprocessing phase converts raw videos into short clips suitable for training. During this phase, we first eliminate broken files and raw videos with outlying attributes. Specifically, we filter out videos with a duration of less than 2 seconds, a bit per pixel (bpp) below 0.02, a frame rate (fps) under 16, and an aspect ratio outside the range $[1/3, 3]$, and those with a "Constrained Baseline" profile.

Then, we detect continuous shots within the raw video and segment it into short clips based on the detection result. For shot detection, we apply the libavfilter from FFmpeg [12] to calculate the scene scores, which quantify visual differences among frames. The shot boundaries are identified at points where the change in scene scores exceeds an empirically determined threshold.

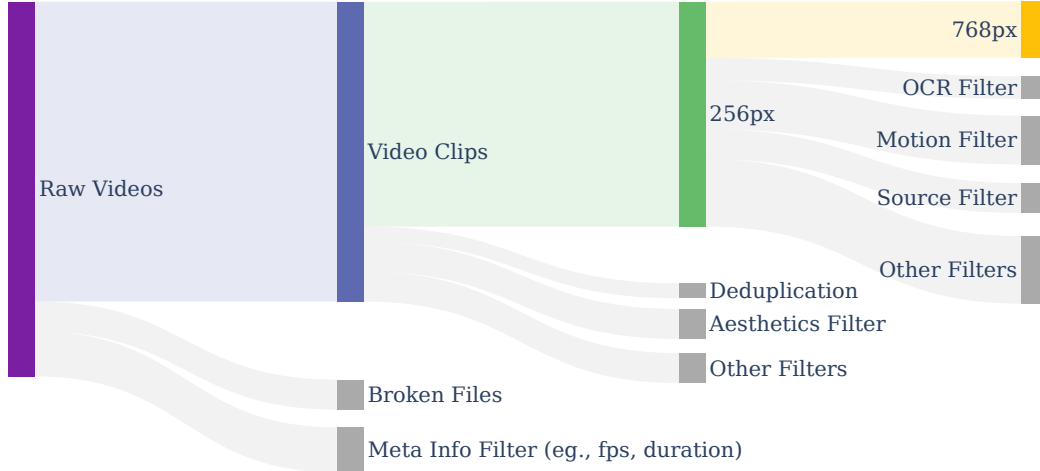


Figure 2: **The hierarchical data filtering pipeline.** The raw videos are first transformed into trainable video clips. Then, we apply various complimentary score filters to obtain data subsets for each training stage.

Finally, we segment the video based on the shot detection results, ensuring that the output clips adhere to specific format constraints: frame rate (fps) below 30, longer dimension not exceeding 1080 pixels, and H.264 codec. Additionally, black borders are removed during this process. For those shots exceeding 8 seconds, we further divide them into multiple 8-second clips, while shots shorter than 2 seconds are discarded.

2.1.2 Score Filtering

To address the various defects in raw data, we develop a bag of complimentary filters, each targeting a specific aspect of data quality. These filters work together as a comprehensive and robust purification system. Typically, each filter evaluates a sample by assigning a score based on its respective criteria, and the filtering intensity is controlled by a threshold. We introduce all the score-based filters in the following sections.

Aesthetic Score. We assess the aesthetic quality of a sample using the CLIP+MLP aesthetic score predictor [36]. For a video sample, we extract the first, middle, and last frame, compute their individual scores, and take the average as the final aesthetic score.

Motion Score. The motion intensity of a video is measured using the VMAF motion score from FFmpeg’s libavfilter library [12]. Videos with extremely low or excessively high motion scores are filtered out to maintain a balanced motion range.

Blur Detection. To evaluate image clarity, we apply the Laplacian operator from OpenCV [3]. A sample is considered blurry if the variance of the Laplacian image falls below a configurable threshold. For video samples, we extract five uniformly spaced frames and adopt a majority voting approach to determine blurriness.

OCR. We detect text bounding boxes using PaddleOCR [2] and compute the total area of bounding boxes with a confidence score above 0.7. Images or videos containing excessive text are discarded to avoid unwanted overlays.

Camera Jitter Detection. We detect camera jitter using Shot Boundary Detection from the PySceneDetect library [6]. A video is identified as having camera jitter if the average frame-to-frame change exceeds a predefined threshold.

For captioning, we utilize the open-source vision-language model LLaVA-Video [43] to annotate 256px videos. We prompt the model to focus on six aspects for a detailed and comprehensive caption, which are 1) main subjects; 2) subjects’ actions; 3) background and environment; 4) lighting condition and atmosphere; 5) camera movement; 6) video style, such as realistic, cinematic, 3D, animation, and so on. For high-resolution 768px training data, we leverage a stronger proprietary model Qwen 2.5 Max [39] to generate more accurate and semantically aligned captions. We find that Qwen 2.5 Max produces fewer hallucinations and delivers better semantic consistency than LLaVA-Video.

2.3 Data Statistics

Aesthetic Score

Score Range	Percentage
[0, 4.5]	14.8%
[4.5, 5]	41.8%
[5, 5.5]	34.6%
[5.5, 6]	7.9%
[6, inf]	1.0%

Duration (s)

Duration Range (s)	Percentage
[2, 4]	32.8%
[4, 6]	19.8%
[6, 8]	47.4%

Aspect Ratio

Aspect Ratio	Percentage
[0.3, 0.5]	13.1%
[0.5, 0.75]	60.8%
[0.75, 1]	5.1%
[1, 2]	20.9%
[2, 3]	0.1%

Text Length

Text Length Range	Percentage
[0, 75]	29.0%
[75, 100]	37.0%
[100, 125]	21.3%
[125, 150]	9.3%
[150, inf]	3.4%

As shown in Fig. 3, the majority of videos have aesthetic scores ranging between 4.5 and 5.5, indicating a generally moderate level of visual appeal. Video durations range from 2 to 8 seconds, with nearly half of the dataset consisting of clips between 6 and 8 seconds, which offer richer temporal information for learning dynamic patterns. The analysis of aspect ratios shows that the majority fall between 0.5 and 0.75, corresponding to the 16:9 format, ensuring adaptability across different formats and enhancing the model’s generalization. Additionally, over 70% of video captions exceed 75 words, providing detailed descriptions that contribute to a more informative training process.

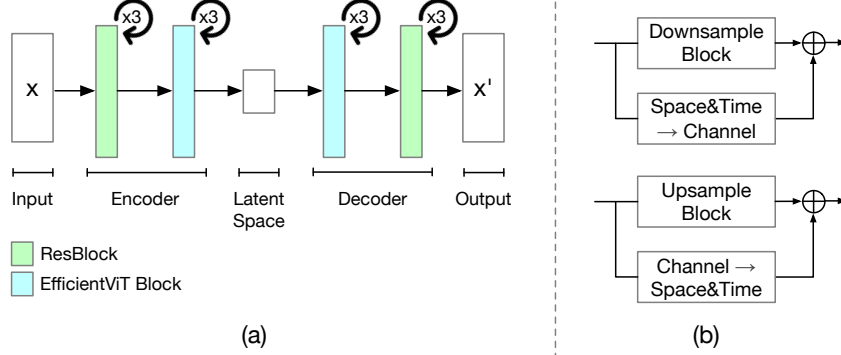


Figure 5: **Architecture of Video DC-AE.** (a) Overview of Video DC-AE: Each block in encoder introduces spatial downsampling, while temporal downsampling occurs at blocks 4 and 5, with a corresponding symmetric structure in the decoder. (b) Residual Connection in Video DC-AE Blocks.

Figure 4 presents a word cloud illustrating the vocabulary distribution in video captions. The captions encompass not only main subjects such as “*person*” and their actions like “*wearing*”, but also background elements (“*background*”, “*setting*”, “*atmosphere*”) and lighting conditions (“*lighting*”). This aligns with the six key aspects of our VLM prompting strategy introduced in Sec. 2.2. Furthermore, the frequent occurrence of “*person*” and “*individual*” suggests that a substantial portion of the dataset features human subjects.

3 Model Architecture

The two key components of a video generation model are the autoencoder and the diffusion transformer. For the autoencoder, our model is initially trained on HunyuanVideo’s VAE and later adapted to our Video DC-AE, whose structure is detailed in Sec. 3.1. The architecture of the diffusion transformer is presented in Sec. 3.2.

3.1 3D Autoencoder

We begin by leveraging the open-source HunyuanVideo VAE [19] as the initial autoencoder for our model. To further reduce both training and inference costs, we develop a video autoencoder with deep compression (Video DC-AE, named after [9]) that improves efficiency while maintaining high reconstruction fidelity.

HunyuanVideo VAE achieves a compression ratio of $4 \times 8 \times 8$, which allows an 8-second, high-resolution (1280×720) video at 16 FPS to be transformed into a latent representation of $32 \times 160 \times 90$. However, even with a patch size of 2×2 , our generation model still needs to process around 115K tokens per training video, resulting in considerable computational demands.

To address the above challenge, we identify a potential redundancy in the spatial dimension and propose to increase the spatial compression ratio to 32 while keeping the same temporal compression ratio at 4. This adjustment effectively reduces the number of spatial tokens while preserving essential motion features in our Video DC-AE. Since our AE training data consists of 32-frame, 256px videos,

Model	Down. (TxHxW)	Info. Down.	Channel	Causal	LPIS↓	PSNR↑	SSIM↑
Open Sora 1.2 [44]	$4 \times 8 \times 8$	192	4	✓	0.161	27.504	0.756
StepVideo VAE [27]	$8 \times 16 \times 16$	96	64	✓	0.082	28.719	0.818
HunyuanVideo VAE [19]	$4 \times 8 \times 8$	48	16	✓	0.046	30.240	0.856
Video DC-AE	$4 \times 32 \times 32$	96	128	×	0.051	30.538	0.863
Video DC-AE	$4 \times 32 \times 32$	48	256	×	0.049	30.777	0.872

Table 1: **Auto-encoder reconstruction performance comparison.** The Video DC-AE highlighted with yellow background is selected for generative model adaptation.

this new compression ratio allows us to train the autoencoder at a latent shape of $8 \times 8 \times 8$, which maintains the same level of information in all dimensions.

Chen *et al.* [9] introduce DC-AE as an effective approach to achieve higher downsampling ratios while maintaining reconstruction quality. While the original DC-AE is primarily optimized for image encoding, we extend its architecture to support video encoding by incorporating temporal compression mechanisms.

As shown in Figure 5(a), our Video DC-AE encoder consists of three residual blocks followed by three EfficientViT blocks [5], with the decoder adopting a symmetrical structure. The first five encoder blocks and the last five decoder blocks function as downsampling and upsampling layers, respectively.

To adapt DC-AE for video encoding, we replace 2D operations (e.g., convolutions, normalization) with 3D operations. Additionally, we introduce temporal compression in the last two downsampling blocks (blocks 4 and 5) of the encoder, and temporal upsampling in the first two upsampling blocks (blocks 2 and 3) of the decoder to reconstruct temporal information effectively.

Moreover, we follow DC-AE to introduce special residual blocks to connect the downsample and upsample blocks. This is because Chen *et al.* [9] identifies gradient propagation issues in these blocks, such that without the residuals, it is especially difficult to train high-compression autoencoders.

As shown in Figure 5(b), our downsample residual blocks follow DC-AE’s pixel-shuffling strategy, redistributing pixels from the spatial and temporal dimensions into the channel dimension, followed by channel-wise averaging to achieve the desired compression. The upsample residual blocks perform the inverse operation by first duplicating channels, and subsequently redistributing them back into the spatial and temporal dimensions to reconstruct the original structure.

We train Video DC-AE from scratch and evaluate its reconstruction quality, with results summarized in Table 1. Our model is compared against OpenSora 1.2 VAE, the recently open-sourced StepVideo VAE [27], and HunyuanVideo VAE [19]. The results demonstrate that Video DC-AE achieves competitive performance, with only minor degradation in LPIPS [42] scores compared to the best-performing HunyuanVideo VAE, while maintaining strong PSNR and SSIM scores.

Furthermore, while the 256-channel version of Video DC-AE offers higher reconstruction quality, we opt for the 128-channel variant in the video generation model to enable faster adaptation due to its reduced channel size (see Section 4.3 for further details).

3.2 DiT Architecture

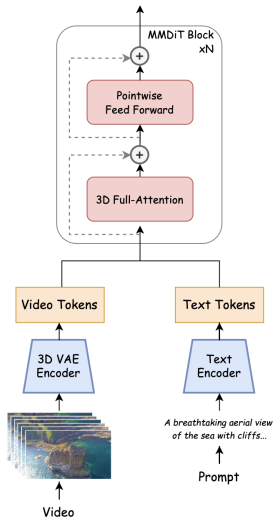


Figure 6: Open-Sora diffusion transformer architecture.

To achieve higher video quality, we employ full attention to capture long-range dependencies effectively. After encoding the input through the autoencoder, we patchify the latent representations to enhance computational efficiency and improve model learning. Sana [40] suggests that a patch size of 1 yields better training stability and finer details in video generation. With Video DC-AE’s high compression ratio, we can afford to reduce the patch size to 1 (i.e., no patch at all), whereas a patch size of 2 is still required when the HunyuanVideo autoencoder is used in our generation model.

Inspired by FLUX’s MMDiT [20], we employ a hybrid transformer architecture that incorporates both dual-stream and single-stream processing blocks. In the dual-stream blocks, text and video information are processed separately to facilitate more effective feature extraction within each modality. Subsequently, single-stream blocks integrate these features to facilitate effective cross-modal interactions. To further enhance the model’s ability to capture spatial and temporal information, we apply 3D RoPE (Rotary Position Embedding) [38], which extends traditional positional encoding to three-dimensional space, allowing the model to better represent motion dynamics across time.

For text encoding, we leverage T5-XXL [10] and CLIP-Large [33], two high-capacity pretrained models known for their strong semantic understanding. T5-XXL captures complex textual semantics, while CLIP-Large improves alignment between text and visual concepts, leading to more accurate prompt adherence in video generation.

An overview of the model architecture is illustrated in Figure 6, and the detailed architectural specifications are presented in Table 2.

Double-Stream Layers	Single-Stream Layers	Model Dimension	FFN Dimension	Attention Heads	Patch Size
19	38	3072	12288	24	2

Table 2: Architecture hyperparameters for Open-Sora 2.0 11B parameter video generation model.

4 Model Training

A commercial-grade video generation model typically requires more than 10 billion parameters and $O(100M)$ training samples. To mitigate the substantial computational cost associated with training, we propose a cost-effective training pipeline, as outlined in Table 3. Our approach consists of three key stages: (1) training a text-to-video model on low-resolution video data, (2) training an image-to-video model on low-resolution video data, and (3) fine-tuning an image-to-video model on high-resolution videos.

Training Stage	Dataset	CP	#iters	#GPUs	#GPU day	USD
256px T2V	70M	1	85k	224	2240	\$107.5k
256px T/I2V	10M	1	13k	192	384	\$18.4k
768px T/I2V	5M	4	13k	192	1536	\$73.7k
Total					4160	\$199.6k

Table 3: **Training Configurations and Cost Breakdown.** This table presents the training configurations across different stages and the total cost for a single full training run, assuming the rental price of H200 is \$2 per GPU hour.

4.1 Efficient Training Strategy

To develop a high-quality video generation model within a constrained budget, our training strategy emphasizes the following four key aspects:

Leveraging Open-Source Image Models Prior research [16, 29, 44] has demonstrated that pre-training on image datasets can significantly accelerate video model training. To avoid the high cost of training an image model from scratch, we leverage an open-source solution. Specifically, we adopt Flux [20], a state-of-the-art text-to-image model with 11 billion parameters, which provides sufficient capacity to generate high-quality videos. We initialize our text-to-video model using Flux and empirically find that, despite it being a distilled model, this initialization is effective for further training.

High-Quality Training Data Inspired by the efficient image training strategies from PixArt [8], we hypothesize that high-quality video data can substantially enhance training efficiency. Consequently, we curate a high-quality subset from a large-scale dataset for low-resolution training. For high-resolution fine-tuning, we impose stricter selection criteria to ensure superior video quality.

Learning Motion in Low-Resolution Training a commercial video generation model is computationally expensive, particularly at high resolutions. To mitigate this cost, we first train on 256px resolution videos, allowing the model to learn diverse motion patterns efficiently. However, we observe that while the model captures motion effectively, low-resolution outputs tend to be blurry. Increasing the resolution significantly improves perceptual quality and human evaluative feedback.

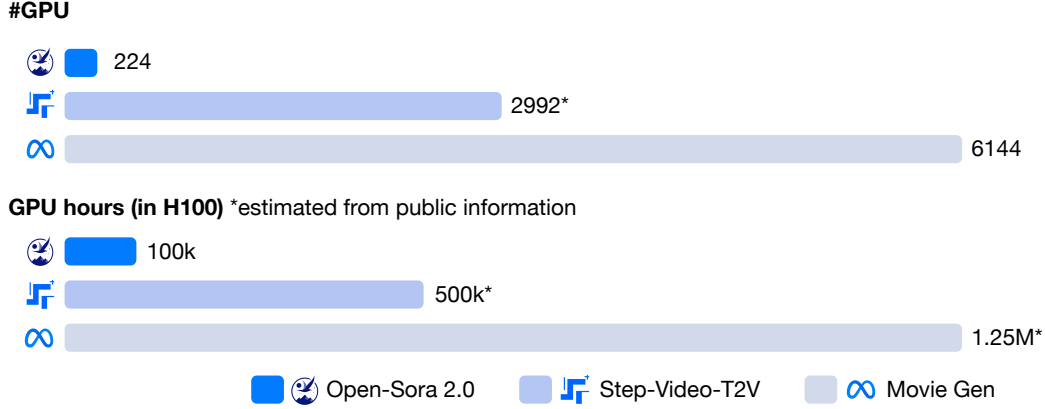


Figure 7: **Training Cost Comparison of Different Video Generation Models.** Values marked with * are estimated based on publicly available information. Our Open-Sora 2.0 is trained at 5–10× lower training cost.

As shown in Table 4 and Table 5, training on a 129-frame video at 768px resolution is 40 times slower than at 256px. This performance gap arises due to the quadratic computational complexity of self-attention mechanisms as the number of tokens increases. To optimize efficiency, we allocate the majority of computational resources to low-resolution training, reducing the need for expensive high-resolution computations.

Image-to-Video Models Facilitate Resolution Adaptation We find that adapting a model from 256px to 768px resolution is significantly more efficient using an image-to-video approach compared to text-to-video. We hypothesize that conditioning the model on a static image allows it to focus more on motion generation, a capability that is well-learned during low-resolution training.

Based on this observation, we prioritize training an image-to-video model at high resolution. During inference, we first generate an image from a text prompt and subsequently synthesize a video conditioned on both the image and the text. During training, text/image-to-video training at low resolution followed by brief fine-tuning on high-resolution videos yields high-quality results with minimal additional training.

With our proposed approach, we successfully constrain the one-time training cost to \$200K, as detailed in Table 3. We estimate the training costs of Movie Gen [32] and Step-Video-T2V [27] based on publicly available information, as summarized in Figure 7. Our model achieves 5–10× lower training costs. We hope that these insights will contribute to reducing the training cost of high-quality video generation models in future research.

4.2 Training Setting

Our training setup is largely based on our previous iteration, Open-Sora 1.2 [44]. We adopt flow matching [23] as our primary training objective and utilize the AdamW [25] optimizer with β values of (0.9, 0.999) and an ϵ value of 1×10^{-15} . No weight decay is applied. The learning rate is set to 5×10^{-5} for the first 40k steps of Stages 1 and 2, followed by a decay to 3×10^{-5} for the final 45k steps. For Stage 3, it is reduced to 1×10^{-5} . Gradient norm clipping is applied with a threshold of 1.

Training Objective Our flow matching approach is similar to that used in Stable Diffusion 3 [11]. We denote the video latent as X_0 , and Gaussian noise $X_1 \sim \mathcal{N}(0, 1)$. The model f_θ takes as input an interpolated latent $X_t = (1 - t)X_0 + tX_1$ and is trained to predict the velocity component $X_0 - X_1$. The corresponding loss function is formulated as:

$$\mathcal{L} = \mathbb{E}_{t, X_0, X_1} [\|f_\theta(X_t, t, y) - (X_0 - X_1)\|],$$